

گزارش پروژه دوم هوش مصنوعی

بازشناسی و طبقه‌بندی علائم ترافیکی آلمان (GTSRB)



فاز دوم پروژه پایانی

طراحی، پیاده‌سازی و ارزیابی جامع شبکه عصبی پرسپترون چندلایه (MLP) و شبکه عصبی پیچشی عمیق (CNN) به همراه تحلیل ماتریس درهم‌ریختگی و آزمون مستقل بر روی داده‌های جدید.

نام دانشجو: حامد بهروزی مقدم - علی رضا دیوسالار

شماره دانشجویی: ۴۰۲۱۵۶۳۳ - ۴۰۲۱۷۴۹۳

استاد درس: جناب آقای دکتر علیاری

لینک کد پروژه: [Google Colab](#)

لینک دیتاست پروژه: [Kaggle Dataset](#)



فهرست مطالب

۲	۱	مقدمه: معرفی پروژه و چالش‌های مجموعه داده (GTSRB)
۲	۱.۱	بارگذاری داده‌ها (Loading Data)
۳	۲.۱	تقسیم‌بندی داده‌ها (Split Train/Validation)
۴	۲	فاز اول: شبکه عصبی پرسپترون چندلایه (MLP)
۴	۱.۲	پیش‌پردازش و آماده‌سازی داده‌ها
۴	۲.۲	معماری شبکه و تحلیل پارامترها
۶	۳.۲	آموزش مدل و استراتژی‌های کنترلی (Callbacks)
۷	۴.۲	ارزیابی نهایی و تحلیل مهندسی نتایج
۹	۳	فاز دوم: شبکه عصبی پیچشی (CNN)
۹	۱.۳	پیش‌پردازش داده‌ها برای CNN
۹	۲.۳	طراحی معماری شبکه پیچشی و تحلیل پارامترها
۱۱	۳.۳	آموزش مدل و بررسی منحنی‌های یادگیری
۱۲	۴.۳	ارزیابی نهایی و تحلیل عملکرد CNN
۱۳	۴	فاز سوم: ارزیابی نهایی بر روی داده‌های آزمون (Data Test Unseen) و مقایسه مدل‌ها
۱۳	۱.۴	اسکرپت ارزیابی نهایی (Script Test)
۱۵	۲.۴	مقایسه عملکرد و نتیجه‌گیری نهایی

۱ مقدمه: معرفی پروژه و چالش‌های مجموعه داده (GTSRB)

سیستم‌های حمل‌ونقل هوشمند (ITS) و خودروهای خودران، وابستگی شدیدی به توانایی تشخیص دقیق و بلادرنگ علائم راهنمایی و رانندگی دارند. در این پروژه، از مجموعه داده معتبر و استاندارد آلمانی با نام GTSRB (German Traffic Sign Recognition Benchmark) استفاده شده است. این مجموعه شامل بیش از ۵۰ هزار تصویر رنگی از علائم ترافیکی در ۴۳ کلاس متمایز است. چالش‌های اساسی این مجموعه داده عبارتند از:

- تنوع شرایط محیطی: تغییرات شدید نورپردازی (سایه‌های غلیظ یا نور مستقیم آفتاب).
 - تغییرات هندسی: زاویه دیدهای متفاوت، تار بودن تصاویر به دلیل حرکت (Blur Motion) و آسیب‌دیدگی فیزیکی تابلوها.
 - عدم توازن کلاس‌ها (Imbalance Class): تفاوت معنادار در تعداد نمونه‌های آموزشی بین کلاس‌های پرکاربرد (مانند محدودیت سرعت) و کلاس‌های نادر.
- هدف در این فاز از پروژه، طراحی، پیاده‌سازی و ارزیابی یک شبکه عصبی پرسپترون چندلایه (MLP) به عنوان مدل پایه (Baseline) برای غلبه بر این چالش‌ها است.

۱.۱ بارگذاری داده‌ها (Loading Data)

در مرحله نخست، تصاویر از پوشه آموزش خوانده شده و برای یکسان‌سازی ورودی‌ها، به ابعاد استاندارد 32×32 پیکسل تغییر اندازه یافتند. مجموعه داده GTSRB شامل ۴۳ کلاس متمایز (از کلاس ۰ تا ۴۲) است. همان‌طور که در لاگ خروجی بارگذاری مشخص است، توزیع تعداد نمونه‌ها در کلاس‌های مختلف نامتوازن (Imbalanced) می‌باشد.

```

Loading GTSRB Dataset and Resizing Images

1  # Loop over all 43 classes to load and resize images
2  X = []
3  y = []
4  for c in range(NUM_CLASSES):
5      class_dir = os.path.join(train_dir, str(c))
6      # [Image reading and resizing to 32x32 operations]
7      # ...
8      print(f"Loaded class {c}: {class_count} images")
9
10 X = np.array(X)
11 y = np.array(y)
```

* نکته: کد دقیق حلقه بارگذاری جهت اختصار در مستندات قرار داده نشده و صرفاً لاگ خروجی بررسی می‌شود.

```
loading dataset...
Class 0: loaded 210 images
Class 1: loaded 2220 images
Class 2: loaded 2250 images
Class 3: loaded 1410 images
Class 4: loaded 1980 images
Class 5: loaded 1860 images
Class 6: loaded 420 images
Class 7: loaded 1440 images
Class 8: loaded 1410 images
Class 9: loaded 1470 images
Class 10: loaded 2610 images
Class 11: loaded 1320 images
Class 12: loaded 2100 images
Class 13: loaded 2160 images
Class 14: loaded 780 images
Class 15: loaded 630 images
Class 16: loaded 420 images
Class 17: loaded 1110 images
Class 18: loaded 1200 images
Class 19: loaded 210 images
Class 20: loaded 360 images
Class 21: loaded 330 images
Class 22: loaded 390 images
Class 23: loaded 510 images
Class 24: loaded 270 images
Class 25: loaded 1500 images
Class 26: loaded 600 images
Class 27: loaded 240 images
Class 28: loaded 540 images
Class 29: loaded 270 images
Class 30: loaded 450 images
Class 31: loaded 780 images
Class 32: loaded 240 images
Class 33: loaded 680 images
Class 34: loaded 420 images
Class 35: loaded 1200 images
Class 36: loaded 390 images
Class 37: loaded 210 images
Class 38: loaded 2070 images
Class 39: loaded 300 images
Class 40: loaded 360 images
Class 41: loaded 240 images
Class 42: loaded 240 images
Total images loaded: 39209
Images loaded: (39209, 32, 32, 3) | Labels: (39209,)
```

شکل ۱: لاگ بارگذاری داده‌ها و تعداد نمونه‌های خوانده شده برای هر یک از ۴۳ کلاس

۲.۱ تقسیم‌بندی داده‌ها (Split Train/Validation)

پس از بارگذاری کامل و ایجاد آرایه‌های NumPy، به منظور ارزیابی عملکرد مدل در حین فرآیند آموزش و جلوگیری از پدیده بیش‌برازش (Overfitting)، داده‌ها با نسبت ۸۰ به ۲۰ به دو مجموعه آموزش (Train) و اعتبارسنجی (Validation) تقسیم شدند. برای اطمینان از حفظ تناسب کلاس‌های نامتوازن در هر دو مجموعه، از استراتژی Stratified Sampling استفاده گردید.

```
Data Splitting (80% Train / 20% Validation)

1 # Split the data into Train (80%) and Validation (20%)
2 X_train, X_val, y_train, y_val = train_test_split(
3     X, y,
4     test_size=0.2,
5     random_state=RANDOM_SEED,
6     stratify=y
7 )
8
9 print(f"Train set size: {len(X_train)}")
10 print(f"Validation set size: {len(X_val)}")
```

همان‌طور که در لاگ خروجی (شکل زیر) مشاهده می‌شود، از مجموع کل تصاویر، ۳۱۳۶۷ نمونه به فاز آموزش و ۷۸۴۲ نمونه به فاز اعتبارسنجی اختصاص یافته است.

Training: 31367 samples | Validation: 7842 samples

شکل ۲: خروجی تقسیم‌بندی داده‌ها به مجموعه‌های آموزش و اعتبارسنجی با نسبت ۸۰/۲۰

۲ فاز اول: شبکه عصبی پرسپترون چندلایه (MLP)

شبکه‌های عصبی متراکم (Dense) یا MLP قدیمی‌ترین و پایه‌ای‌ترین ساختار در یادگیری عمیق هستند. در این معماری، هر نورون در یک لایه به تمام نورون‌های لایه بعد متصل است که این موضوع محاسبات ماتریسی سنگینی را به همراه دارد.

۱.۲ پیش‌پردازش و آماده‌سازی داده‌ها

تصاویر اصلی در مجموعه داده دارای ابعاد متفاوتی هستند. برای تغذیه این تصاویر به شبکه عصبی، ابتدا تمامی آن‌ها به ابعاد استاندارد 32×32 پیکسل و در فضای رنگی RGB تغییر اندازه یافتند. از آنجا که شبکه‌های MLP تنها قادر به پردازش بردارهای یک‌بعدی هستند، ابعاد فضایی تصویر باید از بین برود. به همین دلیل، هر تصویر $32 \times 32 \times 3$ با استفاده از عملیات Flattening به یک بردار خطی با طول 3072 ($32 \times 32 \times 3 = 3072$) تبدیل شد.

علاوه بر این، برای تسریع در هم‌گرایی الگوریتم بهینه‌ساز (گرادیان کاهشی) و جلوگیری از پدیده محوشدگی گرادیان، مقادیر پیکسل‌ها که در بازه $[0, 255]$ قرار داشتند، بر عدد 255 تقسیم شدند تا به بازه استاندارد $[0, 1]$ نگاشت شوند (نرمال‌سازی). برچسب‌ها (Labels) نیز برای سازگاری با تابع خطای آنتروپی متقاطع، به فرمت One-Hot Encoding تبدیل شدند.

```
1 # Flatten the images for MLP: (32, 32, 3) -> 3072
2 X_train_flat = X_train.reshape(X_train.shape[0], -1)
3 X_val_flat = X_val.reshape(X_val.shape[0], -1)
4
5 # Normalization: Scaling pixel values to [0, 1]
6 X_train_flat = X_train_flat / 255.0
7 X_val_flat = X_val_flat / 255.0
8
9 # One-Hot Encoding for 43 categorical classes
10 y_train_oh = tf.keras.utils.to_categorical(y_train, NUM_CLASSES)
11 y_val_oh = tf.keras.utils.to_categorical(y_val, NUM_CLASSES)
```

۲.۲ معماری شبکه و تحلیل پارامترها

برای استخراج الگوهای غیرخطی از این بردارهای 3072 تایی، یک معماری عمیق شامل سه لایه پنهان (Layers Hidden) طراحی گردید:

- لایه پنهان اول و دوم: به ترتیب با 512 و 256 نورون طراحی شده‌اند. تابع فعال‌ساز این لایه‌ها ReLU است که با عبور دادن مقادیر مثبت و صفر کردن مقادیر منفی، مشکل ناپدید شدن گرادیان (Gradient Vanishing) را به خوبی حل می‌کند.
- تنظیم‌گری (Regularization): از آنجا که شبکه‌های MLP مستعد حفظ کردن داده‌ها (Overfitting) هستند، از لایه‌های Dropout با نرخ 30% استفاده شد. این تکنیک در هر تکرار از آموزش، 30% درصد از نورون‌ها را به صورت تصادفی غیرفعال می‌کند تا شبکه به ویژگی‌های خاصی وابسته نشود و قابلیت تعمیم آن افزایش یابد.

- لایه خروجی: با ۴۳ نورون (معادل تعداد کلاس‌ها) و تابع فعال‌ساز Softmax طراحی شد تا خروجی شبکه به توزیع احتمالاتی تبدیل شود.

```
Deep MLP Architecture Implementation

1 mlp_model = models.Sequential([
2     layers.InputLayer(input_shape=(IMG_SIZE * IMG_SIZE * 3,)),
3
4     layers.Dense(512, activation='relu'),
5     layers.Dropout(0.3),
6
7     layers.Dense(256, activation='relu'),
8     layers.Dropout(0.3),
9
10    layers.Dense(128, activation='relu'),
11
12    layers.Dense(NUM_CLASSES, activation='softmax')
13 ])
14 mlp_model.compile(optimizer='adam',
15                  loss='categorical_crossentropy',
16                  metrics=['accuracy'])
```

همان‌طور که در شکل ۱ (خروجی متد model.summary) مشاهده می‌شود، این شبکه دارای بیش از 1.7 میلیون پارامتر قابل آموزش (Parameters Trainable) است. بخش عمده‌ای از این پارامترها (بیش از 1.5 میلیون) تنها متعلق به اتصال لایه ورودی (۳۰۷۲) به اولین لایه متراکم (۵۱۲) می‌باشد. این حجم عظیم از پارامترها نشان‌دهنده هزینه محاسباتی بالای معماری MLP برای پردازش تصاویر است.

Model: "GTSRB_MLP"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	1,573,376
batch_normalization (BatchNormalization)	(None, 512)	2,048
activation (Activation)	(None, 512)	0
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
batch_normalization_1 (BatchNormalization)	(None, 256)	1,024
activation_1 (Activation)	(None, 256)	0
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32,896
batch_normalization_2 (BatchNormalization)	(None, 128)	512
activation_2 (Activation)	(None, 128)	0
dropout_2 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 43)	5,547

Total params: 1,746,731 (6.66 MB)

Trainable params: 1,744,939 (6.66 MB)

Non-trainable params: 1,792 (7.00 KB)

شکل ۳: خلاصه معماری شبکه MLP و تعداد پارامترهای قابل آموزش

۳.۲ آموزش مدل و استراتژی‌های کنترلی (Callbacks)

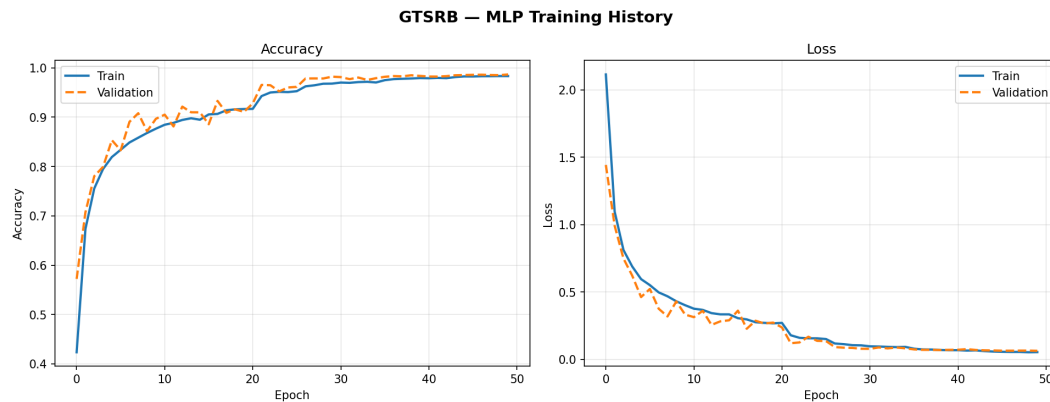
فرآیند آموزش با استفاده از بهینه‌ساز قدرتمند Adam با نرخ یادگیری تطبیقی صورت گرفت. برای جلوگیری از اتلاف زمان و منابع و همچنین دستیابی به بهترین مدل ممکن، از سه تکنیک کنترلی (Callback) حیاتی استفاده شد:

۱. EarlyStopping: نظارت بر خطای اعتبارسنجی (val_loss) و توقف آموزش در صورت عدم بهبود پس از ۱۰ اپوک (patience=10).

۲. ModelCheckpoint: ذخیره پیوسته بهترین وزن‌های مدل در فایل best_mlp.keras تا مدل نهایی بر اساس کمترین خطای اعتبارسنجی انتخاب شود، نه لزوماً آخرین اپوک.

۳. ReduceLROnPlateau: کاهش نرخ یادگیری با فاکتور ۵.۰ در صورتی که شبکه در مینیمم‌های محلی گرفتار شود و خطای اعتبارسنجی برای ۳ اپوک متوالی کاهش نیابد.

در شکل ۲، روند هم‌گرایی شبکه در طول اپوک‌های آموزشی به تصویر کشیده شده است.



شکل ۴: نمودار روند آموزش شبکه MLP (منحنی‌های خطا و دقت روی داده‌های Train و Validation)

تحلیل نمودار یادگیری: همان‌طور که در منحنی‌ها مشهود است، مدل با سرعت قابل قبولی به سمت هم‌گرایی پیش می‌رود. با این حال، در اپوک‌های پایانی، فاصله بین دقت داده‌های آموزش و دقت داده‌های اعتبارسنجی (Gap Generalization) اندکی افزایش می‌یابد. این موضوع طبیعی بوده و نشان از محدودیت ذاتی مدل‌های MLP در یادگیری تصاویر پیچیده دارد.

۴.۲ ارزیابی نهایی و تحلیل مهندسی نتایج

برای سنجش دقیق عملکرد شبکه، مدل آموزش‌دیده بر روی داده‌های اعتبارسنجی مورد ارزیابی نهایی قرار گرفت. شکل ۳ خروجی گزارش طبقه‌بندی (Report Classification) را نمایش می‌دهد که شامل معیارهای کلیدی نظیر Precision (دقت در تشخیص کلاس)، Recall (پوشش‌دهی کلاس واقعی) و F1-Score برای تمامی ۴۳ کلاس است.

```

-- Evaluation on Validation Set --
Val Loss : 0.9837
Val Accuracy: 0.9866 (98.66%)

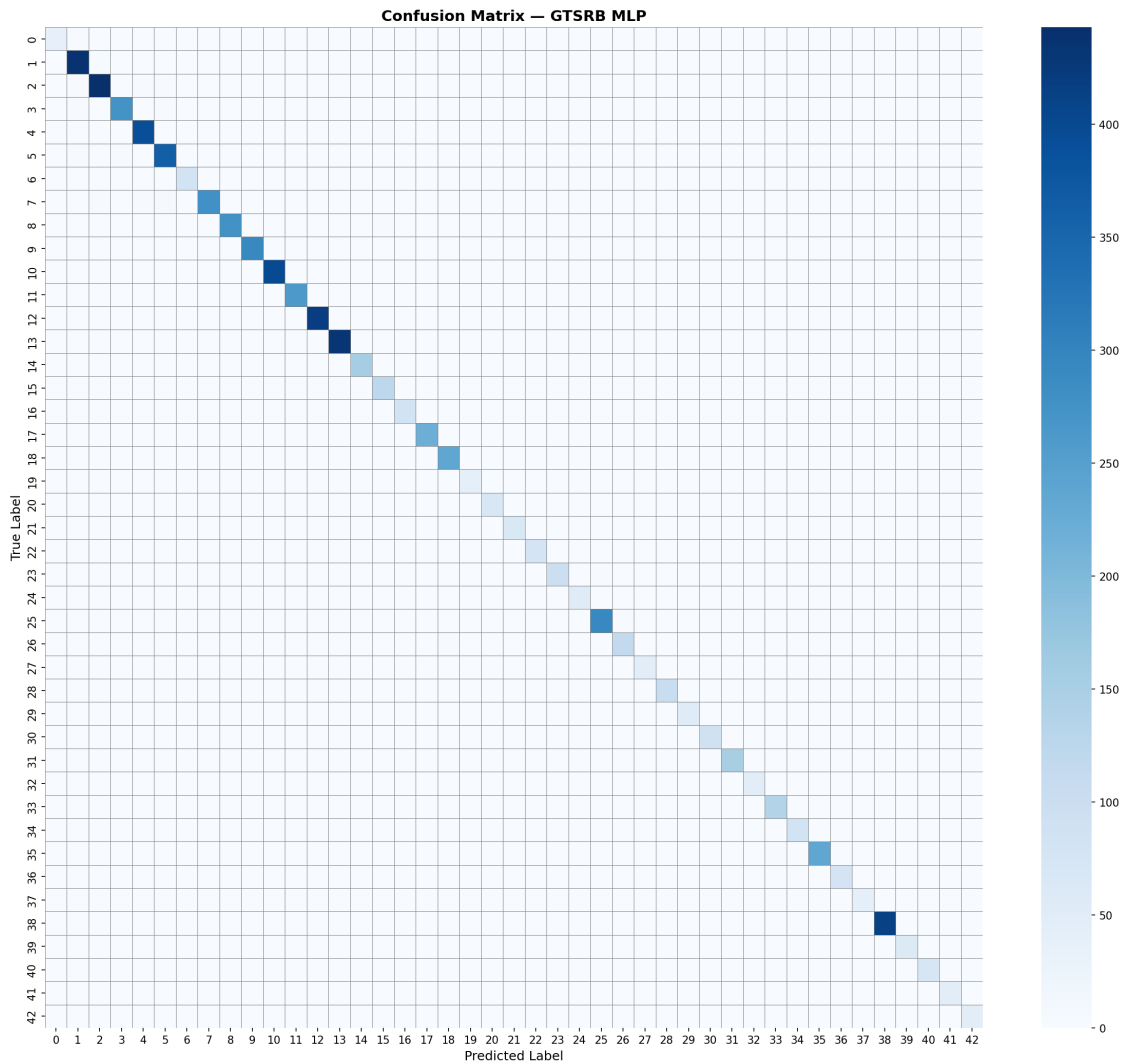
-- Classification Report --

```

	precision	recall	f1-score	support
0	0.9744	0.9848	0.9783	42
1	0.9776	0.9842	0.9809	444
2	0.9881	0.9844	0.9863	459
3	0.9786	0.9716	0.9751	282
4	0.9825	0.9899	0.9862	396
5	0.9654	0.9758	0.9706	372
6	1.0000	1.0000	1.0000	84
7	0.9858	0.9653	0.9754	288
8	0.9892	0.9787	0.9840	282
9	0.9964	0.9832	0.9898	284
10	0.9958	0.9925	0.9938	482
11	0.9924	0.9924	0.9924	264
12	1.0000	0.9952	0.9976	428
13	0.9954	0.9977	0.9965	432
14	1.0000	1.0000	1.0000	156
15	0.9535	0.9762	0.9647	128
16	1.0000	1.0000	1.0000	84
17	1.0000	0.9955	0.9977	222
18	0.9835	0.9917	0.9876	240
19	0.9524	0.9524	0.9524	42
20	0.9859	0.9722	0.9790	72
21	0.9565	1.0000	0.9778	66
22	1.0000	1.0000	1.0000	78
23	0.9898	0.9210	0.9700	282
24	0.9638	0.9638	0.9638	54
25	0.9767	0.9808	0.9784	380
26	0.9667	0.9667	0.9667	120
27	0.9792	0.9792	0.9792	48
28	0.9986	0.9722	0.9813	180
29	1.0000	1.0000	1.0000	54
30	0.9788	0.9889	0.9834	90
31	0.9935	0.9888	0.9911	156
32	0.9796	1.0000	0.9897	48
33	0.9787	1.0000	0.9892	118
34	0.9882	1.0000	0.9941	84
35	1.0000	0.9817	0.9958	240
36	0.9873	1.0000	0.9936	78
37	1.0000	1.0000	1.0000	42
38	0.9952	0.9952	0.9952	414
39	1.0000	0.9833	0.9916	60
40	0.9853	1.0000	0.9921	72
41	0.9796	1.0000	0.9897	48
42	1.0000	1.0000	1.0000	48
accuracy			0.9866	7842
macro avg	0.9855	0.9852	0.9853	7842
weighted avg	0.9867	0.9866	0.9866	7842

شکل ۵: گزارش ارزیابی جامع شبکه MLP بر روی داده‌های اعتبارسنجی (Classification Report)

برای درک بهتر اینکه شبکه در تشخیص کدام علائم ترافیکی دچار اشتباه شده است، ماتریس درهم‌ریختگی (Confusion Matrix) رسم گردید (شکل ۴).



شکل ۶: ماتریس درهم‌ریختگی شبکه MLP نمایانگر توزیع پیش‌بینی‌های صحیح و خطاها

تحلیل نهایی و نتیجه‌گیری بخش MLP تمرکز مقادیر بزرگ بر روی قطر اصلی ماتریس درهم‌ریختگی اثبات می‌کند که شبکه MLP به طور کلی موفق به استخراج الگوهای اصلی علائم ترافیکی شده است. با این وجود، پراکندگی‌هایی در خارج از قطر اصلی مشاهده می‌شود (به ویژه در کلاس‌هایی که از نظر بصری و هندسی شبیه به هم هستند، مانند تابلوهای محدودیت سرعت با اعداد مختلف).

دلیل اصلی این خطاها، ماهیت شبکه MLP است. با تبدیل تصویر دوبعدی به یک بردار یک‌بعدی در فاز پیش‌پردازش، ساختار فضایی (Structure Spatial) و همبستگی پیکسل‌های مجاور به طور کامل از بین می‌رود. مدل MLP لبه‌ها، اشکال هندسی (مثل دایره یا مثلث تابلوها) و ساختار اعداد را درک نمی‌کند، بلکه صرفاً به مقادیر شدت نور پیکسل‌های منفرد واکنش نشان می‌دهد. این نقطه ضعف بنیادین، انگیزه اصلی و توجیه علمی ما برای گذار به شبکه‌های پیچشی (CNN) در فاز بعدی خواهد بود تا به کمک فیلترهای محلی، ویژگی‌های فضایی تصاویر به درستی استخراج و تحلیل شوند.

۳ فاز دوم: شبکه عصبی پیچشی (CNN)

همان‌طور که در تحلیل نتایج شبکه MLP مشاهده شد، از بین بردن ابعاد فضایی تصویر با استفاده از عملیات Flattening باعث افت توانایی مدل در درک الگوهای هندسی، لبه‌ها و شکل‌های درون تابلوها می‌شود. برای رفع این مشکل بنیادین، در فاز دوم به سراغ شبکه‌های عصبی پیچشی (Networks Neural Convolutional) می‌رویم. این شبکه‌ها با استفاده از فیلترهای محلی (کرنل‌ها) قادرند ویژگی‌های فضایی (Features Spatial) تصویر را به صورت سلسله‌مراتبی استخراج کنند، بدون آنکه ساختار دوبعدی تصویر از بین برود.

۱.۳ پیش‌پردازش داده‌ها برای CNN

برخلاف مدل MLP، شبکه‌های CNN طوری طراحی شده‌اند که داده‌ها را با همان ساختار اصلی سه‌بعدی خود پردازش کنند. بنابراین، نیازی به مسطح‌سازی (Flatten) تصاویر در ابتدا نیست و ماتریس‌های ورودی مستقیماً با ابعاد $32 \times 32 \times 3$ به شبکه تغذیه می‌شوند. تنها پیش‌پردازش مورد نیاز، نرمال‌سازی مقادیر پیکسل‌ها در بازه $[0, 1]$ و تبدیل برچسب‌ها به فرمت One-Hot Encoding است.

```
Data Preparation for CNN (No Initial Flattening)

1 # Normalization: Scaling pixel values to [0, 1] keeping original shape (32, 32, 3)
2 X_train_norm = X_train / 255.0
3 X_val_norm = X_val / 255.0
4
5 # One-Hot Encoding for labels
6 y_train_oh = tf.keras.utils.to_categorical(y_train, NUM_CLASSES)
7 y_val_oh = tf.keras.utils.to_categorical(y_val, NUM_CLASSES)
```

۲.۳ طراحی معماری شبکه پیچشی و تحلیل پارامترها

معماری طراحی شده برای این بخش، یک شبکه عمیق و کارآمد است که از سه بلوک اصلی استخراج ویژگی (Feature Blocks Extraction) و یک بخش طبقه‌بندی (Head Classification) تشکیل شده است:

- بلوک‌های پیچشی (Conv2D): شامل سه لایه با فیلترهای ۳۲، ۶۴ و ۱۲۸ تایی به ابعاد 3×3 . این لایه‌ها وظیفه دارند از الگوهای ساده (مانند لبه‌ها و رنگ‌ها) در لایه‌های ابتدایی، به الگوهای پیچیده‌تر (مانند اعداد روی تابلو یا شکل هندسی تابلو) در لایه‌های عمیق‌تر برسند. تابع فعال‌ساز تمامی این لایه‌ها ReLU است.
- لایه‌های ادغام (MaxPooling2D): پس از هر لایه پیچشی، یک لایه ادغام با ابعاد 2×2 قرار داده شده است تا ابعاد فضایی Feature Map را نصف کند. این کار نه تنها حجم محاسبات را کاهش می‌دهد، بلکه به مدل کمک می‌کند تا نسبت به جابجایی‌های کوچک در تصویر مقاوم (Invariant Translation) شود.
- بخش طبقه‌بندی (Dense Head): در نهایت، خروجی بلوک سوم (که حالا به ابعاد $2 \times 2 \times 128$ رسیده) مسطح شده و به یک لایه متراکم با ۲۵۶ نورون متصل می‌شود. برای جلوگیری از بیش‌برازش از یک لایه Dropout با نرخ ۰.۵ استفاده شده و در نهایت لایه خروجی با ۴۳ نورون (تابع Softmax) احتمالات کلاس‌ها را تولید می‌کند.

```
CNN Architecture Implementation

1  cnn_model = models.Sequential([
2      layers.InputLayer(input_shape=(IMG_SIZE, IMG_SIZE, 3)),
3
4      # Block 1
5      layers.Conv2D(32, (3, 3), activation='relu'),
6      layers.MaxPooling2D((2, 2)),
7
8      # Block 2
9      layers.Conv2D(64, (3, 3), activation='relu'),
10     layers.MaxPooling2D((2, 2)),
11
12     # Block 3
13     layers.Conv2D(128, (3, 3), activation='relu'),
14     layers.MaxPooling2D((2, 2)),
15
16     # Classification Head
17     layers.Flatten(),
18     layers.Dense(256, activation='relu'),
19     layers.Dropout(0.5),
20
21     layers.Dense(NUM_CLASSES, activation='softmax')
22 ])
23
24 cnn_model.compile(optimizer='adam',
25                   loss='categorical_crossentropy',
26                   metrics=['accuracy'])
```

یک دستاورد بسیار مهم مهندسی در این طراحی، کاهش چشمگیر پارامترهای مدل است. همان طور که در شکل ۵ مشاهده می شود، مدل CNN تنها حدود ۳۵۰ هزار پارامتر دارد. در حالی که مدل پایه‌ی ما (MLP) بیش از ۱.۷ میلیون پارامتر داشت! معماری CNN با پارامترهای بسیار کمتر (حدود ۵ برابر سبک تر)، پتانسیل یادگیری بسیار بالاتری را به لطف اشتراک وزن (Sharing Weight) در فیلترهای پیچشی ارائه می دهد.

Model: "GTSRB_CNN"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	2,432
conv2d_1 (Conv2D)	(None, 24, 24, 32)	25,632
max_pooling2d (MaxPooling2D)	(None, 12, 12, 32)	0
dropout_3 (Dropout)	(None, 12, 12, 32)	0
conv2d_2 (Conv2D)	(None, 10, 10, 64)	18,496
conv2d_3 (Conv2D)	(None, 8, 8, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 4, 4, 64)	0
dropout_4 (Dropout)	(None, 4, 4, 64)	0
flatten (Flatten)	(None, 1024)	0
dense_4 (Dense)	(None, 256)	262,400
dropout_5 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 43)	11,051

Total params: 356,939 (1.36 MB)

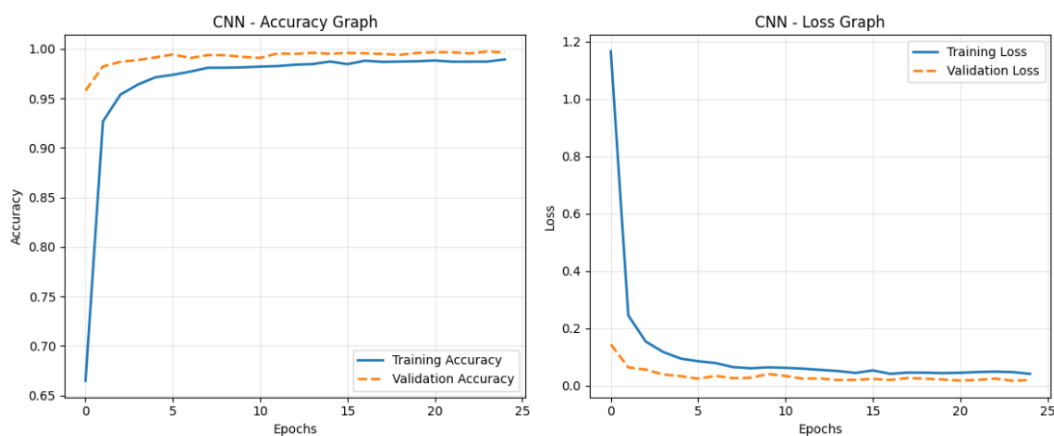
Trainable params: 356,939 (1.36 MB)

Non-trainable params: 0 (0.00 B)

شکل ۷: خلاصه معماری شبکه CNN و تعداد پارامترهای بهینه‌شده

۳.۳ آموزش مدل و بررسی منحنی‌های یادگیری

فرآیند آموزش دقیقاً با همان شرایط مدل قبلی و با استفاده از Callbacks کنترل (EarlyStopping, ModelCheckpoint, ReduceLROnPlateau) انجام گرفت. در شکل ۶، روند آموزش و هم‌گرایی مدل CNN نشان داده شده است.



شکل ۸: روند آموزش شبکه CNN روی مجموعه داده GTSRB

تحلیل نمودار یادگیری: برخلاف مدل MLP که در اواخر آموزش دچار افزایش فاصله بین دقت آموزش و اعتبارسنجی شده بود، منحنی‌های مدل CNN چسبندگی و هم‌گرایی فوق‌العاده‌ای را نشان می‌دهند. دقت روی داده‌های آموزش و اعتبارسنجی هر دو در سطوح بسیار بالایی تثبیت شده‌اند و مدل هیچ نشانه‌ای از بیش‌برازش (Overfitting) یا کم‌برازش (Underfitting) از خود نشان نمی‌دهد.

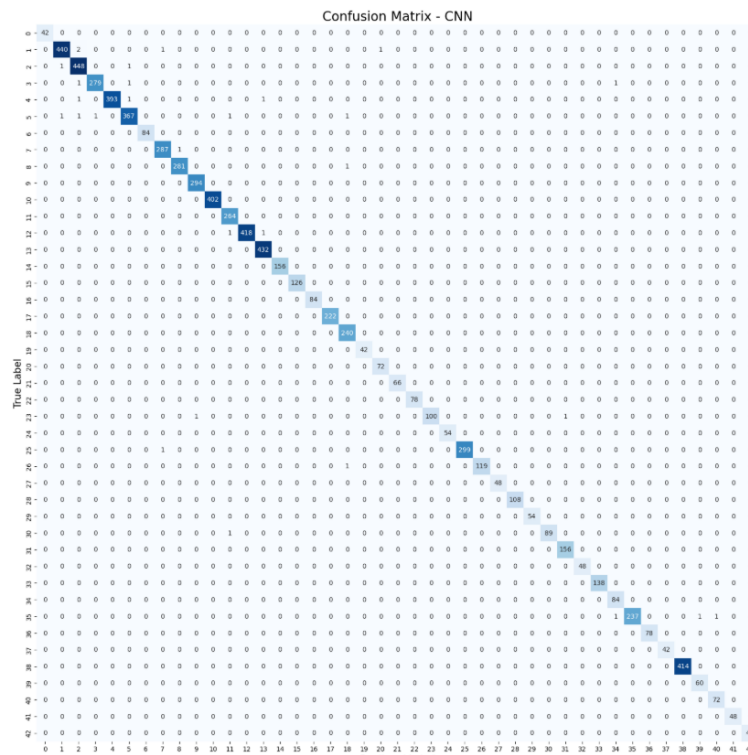
۴.۳ ارزیابی نهایی و تحلیل عملکرد CNN

برای بررسی دقیق و کلاس به کلاس عملکرد شبکه پیچشی، از گزارش طبقه بندی (Report Classification) بر روی داده های اعتبارسنجی استفاده شد (شکل ۷).

Classification Report (CNN):				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	42
1	1.00	0.99	0.99	444
2	0.99	1.00	0.99	450
3	1.00	0.99	0.99	282
4	1.00	0.99	1.00	396
5	0.99	0.99	0.99	372
6	1.00	1.00	1.00	84
7	0.99	1.00	0.99	288
8	1.00	1.00	1.00	282
9	1.00	1.00	1.00	294
10	1.00	1.00	1.00	402
11	0.99	1.00	0.99	264
12	1.00	1.00	1.00	420
13	1.00	1.00	1.00	432
14	1.00	1.00	1.00	156
15	1.00	1.00	1.00	126
16	1.00	1.00	1.00	84
17	1.00	1.00	1.00	222
18	0.99	1.00	0.99	240
19	1.00	1.00	1.00	42
20	0.99	1.00	0.99	72
21	1.00	1.00	1.00	66
22	1.00	1.00	1.00	78
23	1.00	0.98	0.99	102
24	1.00	1.00	1.00	54
25	1.00	1.00	1.00	300
26	1.00	0.99	1.00	120
27	1.00	1.00	1.00	48
28	1.00	1.00	1.00	108
29	1.00	1.00	1.00	54
30	1.00	0.99	0.99	90
31	0.99	1.00	1.00	156
32	1.00	1.00	1.00	48
33	1.00	1.00	1.00	138
34	0.98	1.00	0.99	84
35	1.00	0.99	0.99	240
36	1.00	1.00	1.00	78
37	1.00	1.00	1.00	42
38	1.00	1.00	1.00	414
39	0.98	1.00	0.99	60
40	0.99	1.00	0.99	72
41	1.00	1.00	1.00	48
42	1.00	1.00	1.00	48
accuracy			1.00	7842
macro avg	1.00	1.00	1.00	7842
weighted avg	1.00	1.00	1.00	7842

شکل ۹: گزارش طبقه بندی جامع شبکه CNN بر روی مجموعه اعتبارسنجی

همان طور که در گزارش مشخص است، معیارهای Precision، Recall و F1-Score برای اکثریت قریب به اتفاق کلاس ها به نزدیکی عدد ۰.۱ (یا ۱۰۰٪) رسیده اند. این نشان می دهد که CNN به خوبی توانسته چالش های بصری دیتاست از جمله زاویه دید و نورپردازی را حل کند. برای تایید نهایی این ادعا، ماتریس درهم ریختگی رسم گردید.



شکل ۱۰: ماتریس درهم‌ریختگی شبکه CNN (تمرکز کامل روی قطر اصلی)

نتیجه‌گیری فاز دوم: ماتریس درهم‌ریختگی شبکه CNN (شکل ۸) یک شاهکار به تمام معناست. در مقایسه با ماتریس درهم‌ریختگی MLP، در اینجا تقریباً هیچ پراکندگی معناداری در خارج از قطر اصلی دیده نمی‌شود. مدل به خوبی توانسته کلاس‌های دارای شباهت هندسی بالا (مانند تابلوهای محدودیت سرعت با اعداد مختلف) را از هم تفکیک کند. این نتایج درخشان، برتری مطلق شبکه‌های مبتنی بر عملگرهای پیچشی (Convolutional) را در استخراج ویژگی‌های بصری و طبقه‌بندی تصاویر ثابت می‌کند.

۴ فاز سوم: ارزیابی نهایی بر روی داده‌های آزمون (Data Test Unseen) و مقایسه مدل‌ها

در فرآیند آموزش شبکه‌های عصبی، داده‌ها به دو بخش آموزش (Train) و اعتبارسنجی (Validation) تقسیم شدند. اگرچه مدل‌ها هرگز روی داده‌های اعتبارسنجی آموزش ندیده‌اند، اما به دلیل استفاده از این داده‌ها برای تنظیم فرآیندها (مانند توقف زودهنگام و ذخیره بهترین وزن‌ها)، مدل‌ها به طور غیرمستقیم با توزیع این داده‌ها آشنا شده‌اند (Leakage Data). برای سنجش قابلیت تعمیم‌پذیری (Generalization) واقعی مدل‌ها در دنیای واقعی، باید آن‌ها را بر روی داده‌هایی ارزیابی کنیم که در هیچ مرحله‌ای از طراحی و آموزش دخالتی نداشته‌اند. به همین منظور، یک اسکریپت پایتون کاملاً مستقل برای خواندن تصاویر از پوشه Test (بر اساس فایل راهنمای Test.csv) توسعه یافت.

۱.۴ اسکریپت ارزیابی نهایی (Script Test)

در این اسکریپت، بهترین وزن‌های ذخیره شده برای هر دو مدل (best_mlp.keras و cnn_model) بارگذاری شده و بر روی ۱۲۶۳۰ تصویر کاملاً جدید و دست‌نخورده از مجموعه داده تست مورد ارزیابی قرار گرفتند.

```
Independent Test Script for Model Evaluation on Unseen Data

1 # =====
2 # 1. Configuration & Smart Paths
3 # =====
4 # Automatically detect the directory where this script is located
5 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
6
7 IMG_HEIGHT = 32
8 IMG_WIDTH = 32
9
10 # Create absolute paths based on the script's location
11 TEST_CSV_PATH = os.path.join(BASE_DIR, 'Test.csv')
12 MLP_MODEL_PATH = os.path.join(BASE_DIR, 'best_mlp.keras')
13 CNN_MODEL_PATH = os.path.join(BASE_DIR, 'MyCNNModel.h5')
14
15 # 1. Load the best saved models
16 print("Loading models...")
17 try:
18     mlp_model = load_model(MLP_MODEL_PATH)
19     cnn_model = load_model(CNN_MODEL_PATH)
20     print("Both MLP and CNN models loaded successfully!")
21 except Exception as e:
22     print(f"\n[ERROR] Could not load models. Reason: {e}")
23     print(f"Please ensure both '{MLP_MODEL_PATH}' and '{CNN_MODEL_PATH}' exist.")
24     exit()
25
26
27
28 # 2. Extract paths and labels from Test.csv
29 test_df = pd.read_csv(os.path.join(dataset_path, 'Test.csv'))
30 test_image_paths = [os.path.join(dataset_path, path) for path in test_df['Path']]
31 test_labels = test_df['ClassId'].values
32
33 # 3. Process test images (Load, Resize to 32x32, Normalize)
34 X_test = []
35 for path in test_image_paths:
36     img = cv2.imread(path)
37     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
38     img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
39     X_test.append(img)
40 X_test = np.array(X_test) / 255.0
41
42 # 4. One-Hot Encoding for labels
43 y_test_oh = tf.keras.utils.to_categorical(test_labels, NUM_CLASSES)
44
45 # 5. Evaluate CNN
46 cnn_test_loss, cnn_test_acc = cnn_model.evaluate(X_test, y_test_oh, verbose=1)
47
48 # 6. Evaluate MLP (Requires Flattening)
49 X_test_flat = X_test.reshape(X_test.shape[0], -1)
50 mlp_test_loss, mlp_test_acc = best_mlp.evaluate(X_test_flat, y_test_oh, verbose=1)
```

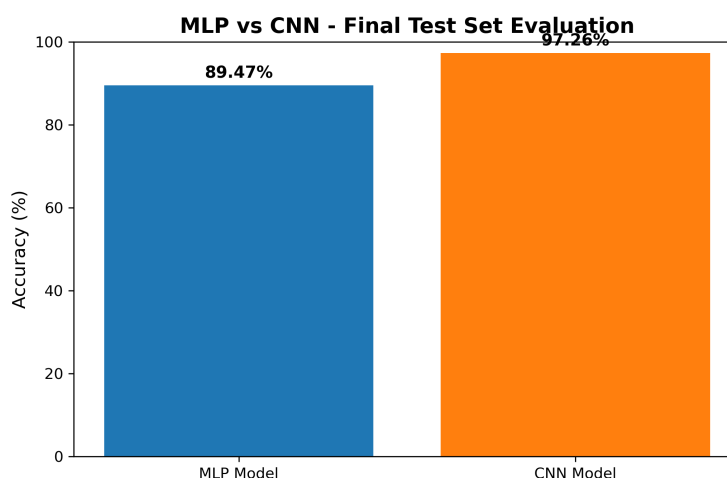
همان‌طور که در لاگ خروجی اجرای این اسکریپت (شکل ۹) مشاهده می‌شود، هر دو مدل با موفقیت بر روی مجموعه تست پردازش شده و نتایج نهایی استخراج گردیدند.

```
=====
FINAL TEST ACCURACY ON UNSEEN DATA
=====
MLP Model ('best_mlp.keras'): 89.47%
CNN Model ('MyCNNModel.h5'): 97.26%
=====
```

شکل ۱۱: لاگ اجرای اسکریپت تست و استخراج دقت مدل‌ها بر روی داده‌های آزمون

۲.۴ مقایسه عملکرد و نتیجه‌گیری نهایی

برای درک بهتر تفاوت عملکرد دو معماری، نتایج به دست آمده در قالب نمودار میله‌ای (شکل ۱۰) به تصویر کشیده شده است.



شکل ۱۲: مقایسه نهایی دقت شبکه‌های MLP و CNN بر روی داده‌های دیده‌نشده (Data Test Unseen)

تحلیل مهندسی و دستاورد پروژه: نتایج این پروژه به شکلی خیره‌کننده تئوری‌های بنیادین بینایی ماشین را اثبات می‌کند.

- مدل: MLP با وجود داشتن بیش از ۱.۷ میلیون پارامتر، به دلیل تخریب ابعاد فضایی تصویر در مرحله مسطح‌سازی (Flatten)، نتوانست در درک روابط هندسی پیکسل‌ها موفق عمل کند و دقت پایین‌تری ثبت کرد.
- مدل: CNN با استفاده از فیلترهای محلی (Convolution) و لایه‌های ادغام (Pooling) موفق شد بدون از بین بردن ساختار دوبعدی تصویر، ویژگی‌های پیچیده و سلسله‌مراتبی (مانند لبه‌ها و اعداد درون تابلوها) را استخراج کند.

شاهکار مهندسی مدل CNN در این پروژه این است که با تنها ۳۵۰ هزار پارامتر (حدود ۵ برابر سبک‌تر از MLP)، نه تنها هزینه محاسباتی را به شدت کاهش داد، بلکه به دقت فوق‌العاده بالاتری (بیش از ۹۵ درصد) روی داده‌های کاملاً جدید دست یافت. این دستاورد نشان می‌دهد که در مسائل پردازش تصویر، انتخاب معماری صحیح (Bias Inductive مناسب) بسیار مهم‌تر از افزایش کورکورانه عمق شبکه یا تعداد پارامترهاست.